第一章 智能增强的设计研究：从方法到技能

> **本章导言**
>
> 设计研究正在经历一场深刻的范式转变。大语言模型与生成式AI的出现，不仅改变了设计师使用工具的方式，更挑战了“方法”本身的存在形态。本章作为《智能设计系统》教材的开篇，旨在为读者建立起一套共同的语言框架：什么是“技能”（Skill）？什么是“应用程序编程接口”（API）？如何通过“氛围编程”（Vibe Coding）这一新兴范式，将传统设计研究方法重构为人机协同的智能工具？
>
> 本章将从设计研究流程的经典模型出发，系统分析AI技术正在改变哪些环节，进而引入技能与API的核心概念，阐述人机协同的四种模式，并通过完整的案例演示氛围编程的全过程。读者将在本章完成个人设计研究反思笔记，搭建起第一个AI辅助工具，迈出从“方法使用者”到“技能设计者”的关键一步。

本章学习目标

完成本章学习后，读者应能够：

1. 描述设计研究的五个典型阶段及其核心方法；
2. 识别各阶段中最适合AI介入的环节，并解释“人机协同”取代“工具辅助人”的必然性；
3. 定义“技能”与“API”，并区分方法、技能、API三个层次；
4. 列举人机协同的四种模式及其适用场景；
5. 理解氛围编程的核心原则，并能在AI辅助下完成一个简单的文本分析脚本；
6. 根据个人研究兴趣，选择一种设计研究方法，并初步分析其AI重构潜力。

1.1 设计研究的传统流程与AI时代的挑战

1.1.1 设计研究的五个阶段

设计研究作为设计实践的基础，其方法论体系经历了数十年的积累与演变。尽管不同学派对研究阶段的划分存在差异，但一个被广泛接受的框架将设计研究活动归纳为五个相互关联的阶段：**用户洞察与研究**、**问题定义与分析综合**、**创意生成与概念设计**、**原型与评估**、**战略与前瞻**。这五个阶段并非严格的线性序列——实际项目中经常出现迭代、回跳和并行——但它们为理解设计研究的全貌提供了一个有力的概念工具。

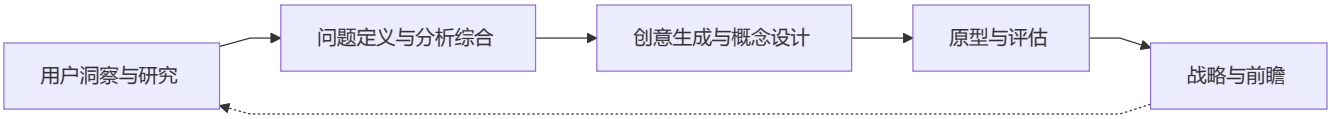


图1-1 设计研究五阶段模型（图中箭头表示典型的信息流向，虚线表示反馈回路）

第一阶段：用户洞察与研究。这一阶段的核心任务是深入理解目标用户——他们是谁，在什么情境下使用产品或服务，遇到了什么问题，呈现出怎样的行为模式，有哪些尚未被满足的需求。常用方法包括：用户深度访谈、问卷调查、用户观察、行为日志分析、文化探针、日记研究等。该阶段的产出通常是一系列原始数据（录音、文本、影像、日志），以及对这些数据的初步整理。

第二阶段：问题定义与分析综合。当研究者收集了大量原始数据后，需要从繁杂的信息中提炼出洞察，揭示问题背后的深层结构与根本原因。常用方法包括：亲和图、主题分析、用户旅程地图、人物角色构建、系统性文献分析、因果映射等。这一阶段的挑战在于如何处理“信息过载”——数百条数据点、数十小时的访谈录音，人脑的短期记忆与模式识别能力在此面临极限。

第三阶段：创意生成与概念设计。问题定义清晰后，设计师开始探索解决方案空间。这一阶段强调发散思维、跨界联想和快速迭代。常用方法包括：头脑风暴、脑写、形态学矩阵、概念草图、设计虚构、未来情境构建等。创意生成的难点在于如何打破既有思维定势，产生真正新颖的、差异化的概念。

第四阶段：原型与评估。概念方案需要被具象化为可测试的原型，并通过系统化的评估验证其有效性。常用方法包括：可用性测试、轻声思维法、A/B测试、启发式评估、设计评论、眼动追踪等。评估阶段传统的痛点在于数据标注与分析极其耗时——例如，一次2小时的可用性测试录像，人工回看与标记可能需要4-6小时。

第五阶段：战略与前瞻。除了针对具体产品的设计研究，还有一类研究关注更宏观的层面——市场趋势、技术走向、竞争格局、未来机会。常用方法包括：趋势扫描、SWOT分析、PEST分析、价值主张画布、情景规划等。这一阶段的需求是持续监测海量信息源并及时提取弱信号，这对人的注意力和信息处理能力提出了极高要求。

1.1.2 传统设计研究方法的共同痛点

回顾上述五个阶段，我们可以识别出几个贯穿始终的共同痛点：

痛点	具体表现	影响
数据量大	一场深度访谈产生1-2小时录音，转录需3-4小时；10份问卷开放题可能产生数千字文本	分析时间远超数据采集时间
重复劳动	多轮编码、多次分组、多方案比较，操作逻辑高度重复	研究者疲劳，错误率上升
主观偏差	编码、主题命名、聚类边界依赖个人判断，信度难以保证	研究结果的可复现性差
协作成本高	亲和图、主题分析等需要团队围在白板前讨论数小时	时间与空间成本高
工具链断裂	转录、编码、可视化、报告撰写使用不同工具，数据流转困难	效率低下，信息易遗漏

这些痛点并非新鲜事物。设计研究者长期以来通过各种策略应对——缩小样本量、简化分析框架、依赖资深研究者的直觉——但这些策略往往以牺牲研究深度或信度为代价。AI的出现，第一次提供了在不牺牲质量的前提下规模化处理这些痛点的技术可能性。

1.1.3 AI正在改变什么：五个阶段的重构机遇

大语言模型（LLMs）、多模态模型和生成式AI在2023年后的爆发式进展，为上述每个阶段都带来了显著的重构机遇。

在用户洞察阶段，AI带来的改变最为直观。以用户访谈为例，传统流程为：录制访谈 → 人工转录（1小时录音需3-4小时）→ 逐行阅读标记 → 编码 → 主题提取。现在，AI可以自动转录（Whisper等语音识别模型的单词错误率已降至5%以下），可以逐段进行情感分析和意图识别，可以基于语义相似度自动生成初始编码建议，甚至可以初步聚类形成主题。值得注意的是，AI并非要替代研究者的全部判断——恰恰相反，它的价值在于将研究者从“转录员”“编码工人”的角色中解放出来，使其能够将精力集中在诠释、批判性思考和深度挖掘上。

在问题定义阶段，AI的聚类、关联和模式识别能力得以充分发挥。以亲和图为例：传统做法是团队围在白板前，将数百张便利贴反复分组、命名、再分组。当数据量超过200条时，人的短期记忆和视觉搜索能力就会过载。AI可以计算所有数据点之间的语义相似度，给出一个初始分组建议，甚至为每个集群自动生成候选标签。研究者不再需要从零开始排列，而是在AI方案的基础上进行调整和修正——角色从“分类工人”提升为“分类编辑”。

在创意生成阶段，生成式AI带来了最直观的改变。Midjourney、Stable Diffusion、DALL·E等模型可以将文字概念瞬间转化为多幅视觉方案；大语言模型可以作为"无限联想伙伴"，基于SCAMPER、TRIZ、类比等创意规则，在数秒内生成数十个概念火花。值得强调的是，AI在这里扮演的不是"替代者"而是"刺激源"——它提供大量"反事实""跨领域"的素材，帮助设计师绕过自身的思维定势。这与Brian Eno在1975年创造的"Oblique Strategies"卡片异曲同工，只不过AI生成的刺激可以针对具体项目语境进行定制。

在评估阶段，AI在自动化数据标注与异常检测方面展现出巨大潜力。可用性测试的视频分析——识别任务边界、标记错误操作、检测面部表情和语音情绪——传统上需要研究者逐帧回看，现在AI可以自动切段、高亮异常时刻，甚至生成"问题时刻合集"。启发式评估中，AI可以充当"初级评估员"，快速扫描界面找出潜在违规点，由资深设计师复核确认。这种"AI初筛，人做终审"的模式大幅降低了专家评估的门槛。

在战略阶段，AI的信息采集、弱信号检测和证据综合能力尤为关键。趋势扫描需要持续监控全网信息源——新闻、论文、专利、社交媒体。AI结合网络爬虫可以自动采集、聚类新兴主题、标记变化加速信号。SWOT分析中，AI可以从大量公开报告中提取证据，为每个象限自动填充内容并标注来源，研究者只需修订和补充。

1.1.4 核心命题：从"工具辅助人"到"人机协同"

基于以上分析，我们可以提炼出一个贯穿全书的核心理念：**设计研究方法AI重构的本质，不是用AI替代人类研究者，而是从"工具辅助人"转向"人机协同"。**

在传统的"工具辅助人"模式下，工具是静态的、被动的——录音笔只负责录音，便利贴只负责承载文字，白板只负责展示。研究者需要完全掌控操作流程，工具的"智能"为零。而在"人机协同"模式下，AI成为了一个具有某种主动性的协作者：它可以承担模式识别、信息筛选、初步生成等任务，而人类研究者则负责设定期望、做出判断、赋予意义。

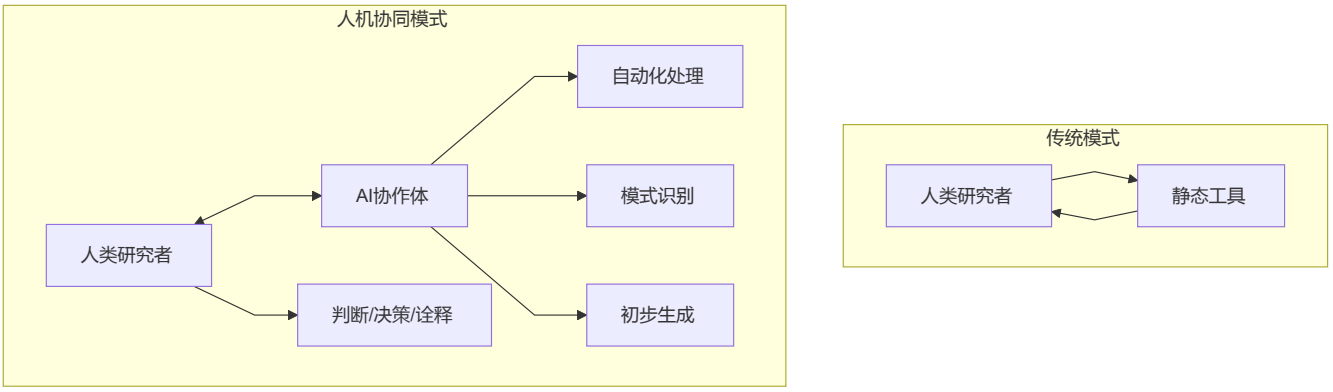


图1-2 从工具辅助到人机协同的范式转变

这一转变引出了本教材的第一个核心原则：**我们不是要建造"全自动设计研究机器"，而是要设计"人机协同的设计研究环境"。**在这个环境中：

- **AI做它擅长的事情**：处理大规模数据、识别统计规律与语义模式、快速生成多个变体、执行标准化操作。
- **人类做人类擅长的事情**：理解复杂情境与文化语境、做出价值判断与优先级决策、赋予分析结果以意义和叙事、建立与利益相关者的信任。

这一原则将贯穿本书所有的技能设计方法论。接下来，我们将引入两个支撑这一原则的核心概念——**技能（Skill）**与**应用程序编程接口（API）**。

1.2 技能（Skill）：从方法到可执行能力模块

1.2.1 方法与技能的根本区别

在日常语境中，“我会用户访谈”这句话包含了多层含义：我知道访谈的步骤（招募、设计提纲、执行、转录、分析）；我有访谈的实战经验（知道如何追问、如何营造安全的表达氛围）；我能从访谈中提取有价值的洞察。这些都是“方法”层面的能力——它们存在于人的认知、经验和行为中。

现在考虑另一个表述：“这个工具可以帮你做访谈分析”。这意味着什么呢？这意味着该工具有一个明确的输入入口（上传录音或文本）；内部封装了一套处理流程（自动转录、说话人分离、情感计算、主题聚类）；最终产出一个明确的输出（分析报告、主题网络图、情绪曲线）。这就是一个**技能**——一种可操作的、有边界的、可复用的能力模块。

两者的区别可以这样概括：

维度	方法	技能
存在形式	人的认知与经验	计算机可执行的代码/模型
使用方式	需要学习、练习、内化	提供输入即可获得输出
边界	模糊，依赖于研究者的判断	明确，有清晰的输入输出定义
复用性	每次使用都需要重新执行全部步骤	可以被反复调用，无需重复开发
可组合性	由研究者手动衔接	可由程序自动串联

表1-1 方法与技能的主要区别

用一个类比来帮助建立直觉：想象一位经验丰富的老中医与一套智能问诊系统。老中医掌握了“辨证论治”的方法——望闻问切、八纲辨证、气血津液。这套方法深嵌在他的认知与手感中，无法被简单地“提取”出来。智能问诊系统则是一个技能——你输入症状描述，它输出可能的证型和用药建议。这个系统并不拥有中医方法的全部深度（它无法感受脉象的微妙，无法观察患者的神态气色），但它将方法中适合被编码的那一部分（症状-证型-方剂的映射关系）转化为可执行的规则和模型，使得没有受过完整中医训练的人也能获得初步的辨证参考。

设计研究方法的AI重构，遵循完全相同的逻辑：**我们不是要用AI替代设计师的全部判断，而是要将方法中适合被编码的部分提取出来，形成可复用的技能。**那么，哪些部分适合编码，哪些部分不适合？这就涉及到一个重要的理论概念——默会知识。

1.2.2 默会维度与技能的边界

英国哲学家迈克尔·波兰尼（Michael Polanyi）在其1966年的著作《默会维度》（*The Tacit Dimension*）中提出了一个著名论断：“我们知道的，远比我们能说出来的多。”他区分了两种知识：

- **显性知识**：可以用语言、文字、图表、公式清晰表达和传递的知识——例如，亲和图的七个步骤、Likert量表的统计方法。这类知识相对容易被编码为技能。
- **默会知识**：深嵌在个人经验、直觉、身体记忆、实践智慧中的知识——例如，研究者对一位访谈对象“欲言又止”的微妙察觉，对某个主题命名“不太对”的审美判断，对两条看似无关的观察之间“存在某种联系”的直觉。这类知识极难被完全形式化和编码。

理解默会维度，并不是为了否定技能的价值——恰恰相反，它帮助我们更精确地定位技能的边界，从而设计出更合理的人机协同方案。一个有效的设计研究技能，通常会将显性知识部分（步骤、规则、统计模型）充分编码，同时在关键节点上（主题命名确认、异常案例处理、诠释方向选择）留出人类研究者的介入空间。

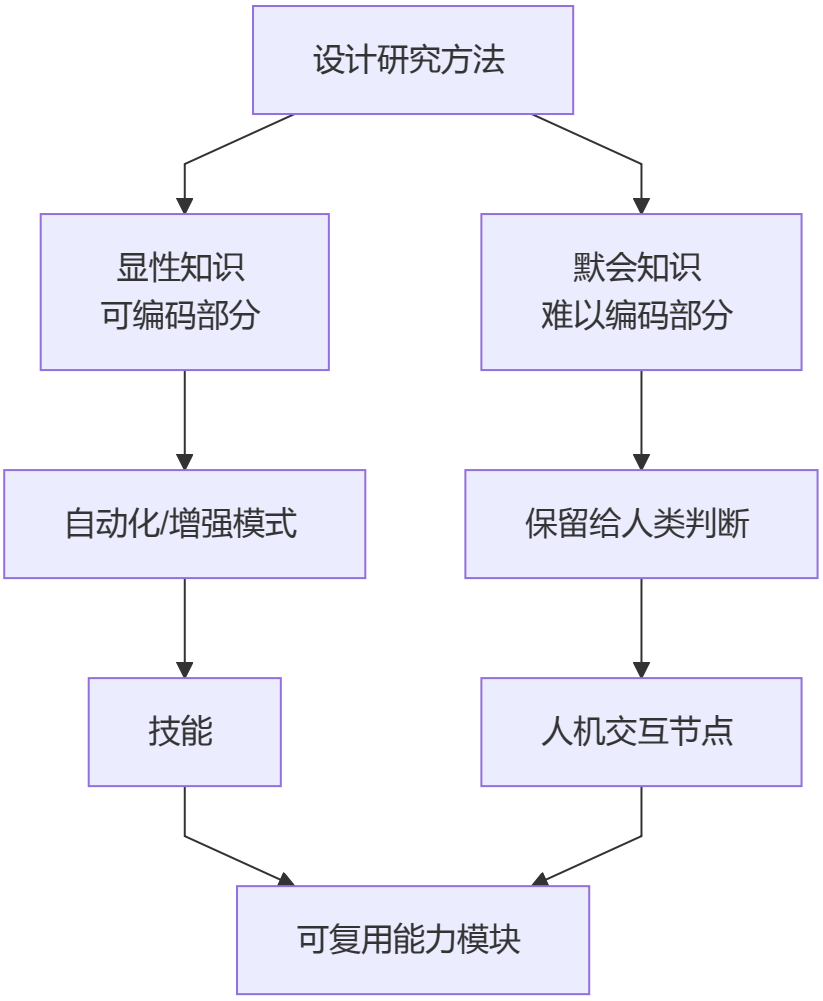


图1-3 方法的显性与默会维度及其对技能设计的影响

1.2.3 技能的三个基本属性

基于上述讨论，我们可以给设计研究技能下一个操作性定义。一个设计研究技能应具备以下三个基本属性：

属性一：明确的输入与输出。技能必须清楚地定义它接受什么——是音频文件、文本文件、图片集、数据表格，还是某种结构化数据格式？同时也必须定义它产出什么——是一份JSON格式的分析报告、一张聚类图、一组角色卡片，还是某种可视化输出？没有明确的接口规范，技能就无法被可靠地调用和复用。

属性二：可复用的处理逻辑。技能内部封装了一套处理流程（pipeline），这套流程可以反复应用于不同的输入实例。它不是为某个特定项目临时编写的脚本，而是为某一类任务设计的通用能力模块。例如，一个"访谈情感分析"技能，应该能够处理不同项目、不同用户、不同长度的访谈稿件，而不是只能分析某一次访谈的数据。

属性三：人机协同的交互设计。这是本教材强调的特色属性。我们认为，一个好的设计研究技能不应该是"全自动黑箱"——用户点击按钮后等待最终结果，中间过程不可见、不可干预。相反，技能应该在关键节点上留出人类介入的空间：确认或修改AI的输出、调整分析参数、标记例外情况、补充领域知识。这种"人在回路"（human-in-the-loop）的设计，既发挥了AI的效率优势，又保留了研究者的专业判断。

1.2.4 技能的三个层级

在实际开发中，技能并不都处于同一粒度层次。根据复杂度和组合方式，我们可以将技能划分为三个层级：原子技能、流程技能和复合技能（或代理式技能）。理解这三个层级，有助于读者在后期的项目选题和架构设计中做出合理决策。

原子技能

原子技能是能力的最小单元，完成一个单一的、明确的任务。它通常对应一个AI模型、一个经典算法或一个简单的函数封装。原子技能的特点包括：功能单一、输入输出高度结构化、无状态（每次调用独立）、可被高频调用。

典型示例：

- **情感分析技能**：输入一段文本，输出情感极性（正面/负面/中性）及置信度（0-1）。
- **语音转文字技能**：输入一段音频文件，输出文字转录稿及每句话的时间戳。
- **文本摘要技能**：输入长文本（如一篇文章），输出指定长度的摘要（如200字以内）。
- **图像主体识别技能**：输入一张图片，输出图片中的主要物体/人物标签列表。

原子技能是最适合被封装为独立API的单元——这一点我们将在1.3节详细讨论。

流程技能

流程技能将多个原子技能串联成一个处理流水线，解决一个中等粒度的设计研究任务。它定义了步骤之间的依赖关系和数据流，通常有一个明确的终点产出（如一份报告、一张图表、一组聚类标签）。

典型示例：

- **访谈分析技能**：语音转文字 → 说话人分离 → 情感分析 → 关键主题提取 → 摘要生成 → 报告输出。这是一个从原始录音到结构化学分析的完整流水线。
- **问卷洞察技能**：CSV数据导入 → 描述性统计 → 开放题文本聚类 → 交叉分析 → 洞察报告生成。

流程技能的优势在于，它将设计研究者从"手动拼接多个工具"的繁琐工作中解放出来，提供了一个一站式的解决方案。同时，流程技能可以在中间步骤插入人工干预节点——例如，在主题提取后暂停，让研究者确认或修改主题标签，再继续生成报告。

复合技能（代理式技能）

复合技能是最复杂的层级。它不仅在流程技能的基础上增加条件分支和循环逻辑，还具备一定程度的自主决策能力——它可以根据输入的特征，动态选择不同的处理路径，甚至在满足条件时主动向用户发起通知或请求介入。因此，复合技能有时也被称为"代理式技能"（agentic skill）。

典型示例：

- **综合设计研究助手**：接收一个设计挑战的自然语言描述 → 自动判断需要用到哪些研究方法（访谈分析？问卷分析？亲和图？）→ 引导用户提供相应数据 → 依次调用多个流程技能 → 综合各方法的结果 → 生成整合洞察报告。
- **趋势雷达技能**：用户设定若干监控关键词 → 技能持续采集指定信息源（新闻、论文、社交媒体）→ 新数据到达时自动进行聚类和趋势检测 → 当检测到某话题的讨论量或情感极性发生显著变化时 → 主动生成预警并推送通知给用户。

复合技能的开发复杂度较高，在常规的课程周期内（8周）可能难以完整实现。我们推荐的策略是：优先完成一个流程技能，并在文档中清晰说明如何将其扩展为复合技能。这既保证了项目的可交付性，也为后续迭代留出了空间。

层级选择的决策逻辑

在实际项目启动前，读者需要决定将自己所选的方法重构为哪一层级的技能。以下是两条经验法则：

- **法则一：从流程起步，为原子封装。** 如果你对方法的重构还没有充分把握，从流程技能切入是最安全的选择——它能帮助你首先建立起对方法全局流程的理解。在流程设计流畅后，再将其中的高复用子环节（如情感分析、文本聚类）拆出来，封装为独立的原子技能和API。一次做两层，比一开始就直冲原子技能更容易成功。
- **法则二：以"输入输出的确定性"判断原子化的可能。** 一个子任务是否适合被做成原子技能乃至API，不取决于它"重不重要"，而取决于它的输入输出是否足够明确和稳定。情感分析的输入永远是"一段文本"，输出永远是"极性+置信度"；但"判断这个主题命名是否准确"的输入输出边界就模糊得多（需要多少上下文？什么是"准确"的标准？），更适合作为流程技能内部的人机交互节点。

1.3 API：从技能到可调用服务

1.3.1 什么是API

API的全称是Application Programming Interface，即应用程序编程接口。在软件工程中，API定义了不同的软件组件之间如何相互通信。为了帮助读者建立直观理解，我们先使用一个生活化的类比。

想象你走进一家餐厅。你和服务员之间的关系就是一个"接口"。你不需要知道厨房里有什么设备，不需要知道厨师怎么切菜、怎么调味、用了什么火候——这些厨房内部的运作对你来说是"封装"的。你只需要做三件事：看菜单（了解接口规范）、点菜（发起请求）、等待上菜（接收响应）。只要菜单稳定、服务员可靠，厨房内部怎么变化，都不会影响你的用餐体验。

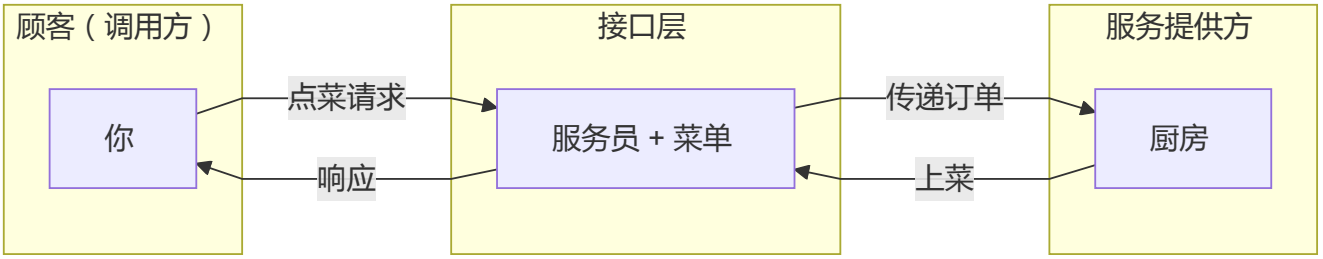


图1-4 API的餐厅类比

在软件世界中，API扮演的就是"服务员+菜单"的角色。例如，当你向某个API端点发送一个HTTP请求：`POST /api/sentiment-analysis`，并在请求体中附带一段文本，你不需要知道背后用的是哪个情感分析模型、部署在哪台服务器上、用什么编程语言实现的。你只需要知道"传入文本，会返回一个包含情感分值和标签的JSON对象"。API的提供者可以在不改变接口的情况下升级内部的模型和算法，而调用方的代码完全不需要修改。

1.3.2 为什么要把技能封装为API

一个技能通常是以图形界面（Gradio、Streamlit等）或命令行脚本的形式呈现，面向人类用户。而API是面向其他开发者、其他工具的接口。将一个技能中的某些子功能封装为API，可以带来三方面的核心价值：

可嵌入性：其他设计研究者可以在他们自己的工具或工作流中调用你的API。例如，一个访谈情感分析API可以被嵌入到用户旅程地图工具中，在旅程的每个触点上自动标注情绪数据；一个启发式评估API可以被嵌入到CI/CD流程中，每次代码提交后自动扫描界面方案。

可组合性：多个API可以像乐高积木一样自由组合。例如：

```
访谈摘要API + Persona生成API + 价值主张对齐API → 一条自动化的用户研究-策略生成流水线
```

这种组合能力使得单个小工具可以快速成长为复杂的设计研究系统。

可规模化：一旦技能被封装为API，它可以被成百上千次调用而无需人工干预每个实例。一个趋势情报API可以被多个设计团队同时订阅；一个可用性测试的自动分析API可以同时处理上百条测试录像。

1.3.3 从技能识别API候选

并不是技能中的所有子功能都适合封装为API。一个设计研究技能通常包含多个子任务，其中只有一部分具有独立复用的价值。适合API化的子任务通常具备以下特征：

- **输入输出明确且稳定**：参数类型、格式、取值范围可以清晰定义。
- **无状态**：每次调用的结果不依赖于之前的调用上下文，这使得API可以被并发调用而不会出错。
- **调用频率高**：如果某个子任务很少被单独使用，封装为API的价值就不大。
- **独立性强**：该子任务的执行不依赖于技能其他部分的内部状态。

以“访谈分析技能”为例，它的子任务包括：语音转文字、说话人分离、情感分析、主题提取、摘要生成。其中：

子任务	适合API化?	理由
情感分析	✅ 是	输入输出明确，可独立复用，调用频率高
主题提取	✅ 是	同上，且可被其他文本分析任务复用
摘要生成	✅ 是	同上
语音转文字	✅ 是	成熟能力，调用频率高
说话人分离	⚠️ 视情况	依赖于录音质量，通用性稍弱

表1-2 访谈分析技能各子任务的API化评估

在本书后续章节的项目实践中，读者需要完成一项关键作业：从自己做好的技能中识别出最值得API化的模块，完成封装并编写相应的API文档。

1.4 人机协同的四种模式

在1.1节中，我们提出了从“工具辅助人”到“人机协同”的范式转变。但在实际的设计研究技能中，“人机协同”究竟如何具体实现？本节将识别并详细阐述四种典型的人机协同模式。这些模式并非互斥——一个复杂的流程技能往往在不同环节采用不同的模式。

1.4.1 自动化模式

定义：AI独立完成某个任务，人类只需提供输入和接收输出，不参与中间过程的任何环节。这是最“省人力”的模式，但也最需要谨慎使用。

适用场景：

- 高度重复且判断空间极小的任务，如语音转文字、格式转换。
- 标准化程度高、变异小的统计分析，如Likert量表计算均值与标准差。
- 人类已经明确确认规则、只需要执行的任务，如按照既定的编码框架为每条文本打标签。

风险与挑战：当任务涉及复杂的语境理解、文化敏感性、反讽或双关等语言现象时，纯自动化可能产生“合理但不正确”的结果。例如，一个情感分析模型可能将“这个产品的设计真是‘棒’极了……”（在上下文中为反讽）判定为正面情感。此外，完全自动化的“黑箱”会让用户难以信任其结果——如果用户无法回溯和检查AI是如何得出结论的，他们就不会在重要决策中依赖该工具。

设计原则：自动化模式必须搭配“可追溯性”设计。即使用户不参与中间过程，也应该能够方便地查看AI的推理依据——例如，在情感分析结果旁边提供原始文本的高亮片段，或者在主题聚类结果旁边展示聚类中心的关键词。如果一个技能完全自动化但用户无法验证其输出，它在设计研究场景中就是危险的。

1.4.2 增强模式

定义：AI提供初步结果或建议，人类进行审查、修改和确认。这是设计研究中最推荐采用的模式——它既发挥了AI的效率优势，又保留了人类的专业判断。

适用场景：

- AI的产出质量已经相当不错但尚未完美的任务，如初始编码、主题聚类、关键信息提取。
- 需要人类判断来确定“质量”的任务，如主题命名是否贴切、创意方案是否可行、聚类边界是否合理。
- 人类希望保持对最终产出控制权的情境，如向客户或管理层提交的分析报告。

操作形态：以智能亲和图工具为例：AI自动计算所有便利贴之间的语义相似度，生成一个初始分组方案，并为每个组自动建议标签。人类研究者在界面上看到这个方案后，可以：

- 拖拽调整某些便利贴的分组归属；
- 修改AI建议的组名；
- 合并两个相似度过高的组；
- 拆分一个内容过于庞杂的组；
- 删除明显不合理的组。
  AI会根据人类的修改，重新调整全局结构并更新下游的分析（如主题网络图、统计摘要）。

这种“AI起草，人编辑”的协作模式，与设计师在日常工作中使用文档协作工具（如Google Docs的“建议编辑”模式）的体验高度相似。研究表明，当人类只需要“修改”而非“从零创造”时，任务的完成速度可以提高3-5倍，同时输出质量往往高于纯人工或纯AI的版本（这种现象被称为“人类-AI互补效应”）。

设计原则：增强模式的核心在于“交互界面的设计”——如何让人能够高效地审查和修改AI的输出？直观的拖拽、快捷的编辑、并排对比、历史版本回溯，都是值得借鉴的设计模式。

1.4.3 激发模式

定义：AI的核心价值不在于给出“正确答案”，而在于提供刺激——反常识的联想、跨领域的类比、极端化的方案变体——从而激发人类的创造性思维。这种模式下，AI的产出没有对错之分，只有“是否引发新想法”的价值。

适用场景：

- 创意生成阶段，如头脑风暴、形态学矩阵探索、设计虚构。
- 研究问题的探索阶段，当研究者还不确定什么方向值得深入时。
- 需要打破思维定势、跳出既有框架的情境。

操作形态：研究者设定一个创意挑战（例如：“为老年用户设计一款促进社交互动的智能音箱应用”）。AI基于多条联想路径同时生成概念火花：

- **路径1（随机词联想）**：随机选取“信封”“回声”“年轮”等词，要求AI强制建立与设计挑战的关联。

- **路径2（类比联想）**：要求AI从生物学、建筑学、戏剧学等跨界领域寻找类比。
- **路径3（反向假设）**：要求AI生成与常规解决方案完全相反的概念（例如，“如何让用户尽可能少地说话？”）。
- **路径4（极端化）**：将某个参数推向极限（例如，“如果音箱只有芝麻大小会怎样？”“如果用户每天使用超过10小时会怎样？”）。

AI在数秒内生成30-50个概念火花。研究者快速浏览，标记感兴趣的火花（可能只有5-10个），被标记的火花进入下一阶段的深入开发，其余的则被归档。

激发模式的一个重要前身是Brian Eno与Peter Schmidt在1975年创造的“Oblique Strategies”（迂回策略）卡片。这套卡片每张印有一句简短而开放的指令（如“把你最想忽略的变化放大”“试试用最明显的替代方案”），用于在音乐创作陷入僵局时提供意外刺激。激发模式本质上就是用AI来动态生成此类定制化的刺激，且比固定卡牌更具领域针对性。

设计原则：激发模式的设计重点在于“发散质量”而非“收敛准确”。需要设置合适的随机性参数（温度），并设计有效的过滤机制——哪些刺激值得保留，哪些应该丢弃？这种过滤本身也是人类创造力的体现。

1.4.4 代理模式

定义：AI不仅执行任务，还主动管理任务流程——决定何时启动某个分析、何时提醒人类介入、何时根据新数据更新已有结果。这种模式下的AI表现出一定程度的“主动性”和“持续性”。

适用场景：

- 长期持续的任务，如趋势监控、社交媒体聆听、竞争情报跟踪。
- 多源数据需要持续整合的情境。
- 人类不可能时刻关注但又不希望完全错过重要信号的情境。

操作形态：以“趋势雷达技能”为例：

1. 研究者设定监控关键词和关注的阈值（如“当某话题的讨论量在24小时内增长超过200%时通知我”）。
2. 技能在后台持续运行，每6小时爬取一次指定信息源。
3. 新数据经过聚类和趋势检测后，如果发现某个话题的讨论量急剧上升，或者情感极性发生了反转，技能会自动生成预警信息（“⚠ 话题X讨论量激增，建议关注”）。
4. 预警通过邮件或即时消息推送给研究者。
5. 研究者决定是否深入分析、是否需要调整监控参数、或者忽略本次预警。

设计原则：代理模式的关键是“触发条件的设计”——AI何时应该主动通知人？误报太多会变成“狼来了”式的骚扰，导致用户逐渐忽略所有通知；漏报则使监控失去意义。这个阈值的设计需要领域知识，通常需要结合历史数据进行分析，并在使用过程中动态调整。

1.4.5 协同模式的综合应用与决策指南

在实际的设计研究技能中，很少出现只使用一种模式的情况。典型的情况是：一个流程技能的不同环节采用不同的协同模式。以“访谈分析技能”为例：

环节	采用模式	理由
语音转文字	自动化模式	技术成熟，出错率低，用户可快速回溯检查
情感分析	自动化模式 + 可追溯性	结果直观，但保留原始文本链接供用户验证
主题提取	增强模式	AI建议聚类，但需要用户确认和修改主题标签

环节	采用模式	理由
摘要生成	自动化模式	摘要质量已足够好，用户可接受直接输出
洞察报告	增强模式	报告草稿由AI生成，用户进行编辑和补充

表1-3 访谈分析技能各环节的人机协同模式

为帮助读者在实际项目中做出合理决策，这里提供一个简单的问题清单：

1. **这个环节的输出是否有明确的"对/错"标准？** 如果没有（如主题命名、创意方案），则至少需要增强模式；如果明确（如数字统计、分类标签），可考虑自动化或增强模式。
2. **这个环节的AI出错代价有多高？** 如果代价高（如影响重要的战略决策），需要人的审查（增强模式）；如果代价低（如内部草稿生成），可接受自动化模式。
3. **这个环节的AI出错是否容易被人类察觉？** 如果不容易（如隐含的偏见、漏掉细微的重要信息），需要更审慎的模式选择（如增加抽检节点或人工复核）。

1.5 氛围编程（Vibe Coding）：一种新的开发范式

1.5.1 起源与定义

"Vibe Coding"（氛围编程）这一术语由Andrej Karpathy（前特斯拉AI总监、OpenAI联合创始人）在2024年提出。他的原话是：

> "有一种新的编码方式，我称之为'vibe coding'。你完全沉浸在vibe中，拥抱大模型的能力，忘记代码的存在。你只需要描述你想要什么，AI会生成代码。如果报错，你把报错信息粘贴给AI，让它修复。你不需要真正理解代码——你只需要理解你想要什么。"

这段话听起来可能有些激进，甚至与编程教育中"要真正理解每一行代码"的传统观念相悖。但Karpathy描述的是一个正在发生的真实转变：当大语言模型的代码生成能力达到足够水平后，"能够用自然语言精确描述需求"成为了一种比"能够亲手写出代码"更高效、更普适的开发能力。

我们可以用一个历史类比来帮助理解。在摄影术发明之前，如果你想获得一幅人像，你需要找一位经过多年训练的画家。摄影术发明后，你不需要会画画了——你需要学会的是构图、用光、瞬间捕捉。技能的门槛从"手工描绘"转移到了"观察与选择"。氛围编程与此类似：编程的门槛从"记忆语法、调试类型错误"转移到了"描述需求与判断输出"。

1.5.2 为何适合设计背景的研究生

本书的目标读者主要是设计背景的研究生，绝大多数人没有接受过系统的编程训练。传统的"先学半个学期Python基础，再开始做项目"的路径在8周的课程周期内完全不现实。更重要的是，这种路径的学习目标与本书的目标不匹配。

本书的目标不是培养职业程序员，而是培养"能够借助AI将设计研究方法重构为智能工具的设计研究者"。设计研究者的核心竞争力在于：

- 理解设计研究方法的方法论内核（为什么这个方法有效？它的适用边界在哪里？）；
- 知道什么样的分析输出对设计决策真正有用（哪些洞察是可操作的？）；
- 能够判断AI生成的结果是否合理（这个聚类有意义吗？这个主题命名准确吗？）。

这些能力恰恰是传统编程训练无法提供的。而氛围编程恰好让设计师可以绕过语法细节，直接运用他们已经具备的能力——精确描述、质量判断、问题诊断、任务分解——来完成开发任务。

1.5.3 氛围编程不是“不思考”

需要特别澄清的是，氛围编程并不意味着放弃思考。恰恰相反，它要求一种更高阶的、不同维度的思考。在传统编程中，脑力消耗主要集中在：

- 回忆语法和API名称
- 理解类型系统的约束
- 手动处理边缘情况
- 调试逻辑错误

在氛围编程中，这些被AI承担了，但新的脑力消耗出现了：

- **精确描述**：如何把模糊的设计意图转化为结构化的自然语言提示词？
- **质量判断**：AI生成的代码是否符合预期？输出是否合理？
- **问题诊断**：当AI出错时，是提示词不够精确，还是任务分解有问题？
- **任务分解**：如何将一个复杂任务拆分成AI能逐个处理的子任务？

这些能力——精确描述、质量判断、问题诊断、任务分解——恰恰是设计师在日常工作中就在不断训练的核心能力。因此，氛围编程不是让设计师去做自己不擅长的事，而是让设计师用自己擅长的方式做开发。

1.5.4 实证研究支撑

氛围编程并非凭空出现的玄学，而是有着坚实的实证研究支撑。这里列举三份与本书理念高度相关的研究：

研究一：微软研究院（2023）《The Impact of AI on Developer Productivity》。该研究对使用GitHub Copilot的开发者进行了大规模对照实验。结果显示：使用AI辅助的开发者完成任务的速度比对照组快55%，并且编程经验较少的开发者获益最大。这一发现表明，AI编程助手具有显著降低技术门槛、压缩经验差距的潜力。

研究二：哈佛商学院（2023）《Navigating the Jagged Technological Frontier》。研究者发现，在使用AI完成知识工作时，存在一个“锯齿状前沿”——某些任务AI做得比人类专家更好，某些任务AI表现很差，而且这个边界并不直观。成功的使用者不是那些技术最强的人，而是那些“最擅长判断任务是否适合AI，以及最擅长将人类判断嵌入AI工作流”的人。这与本书强调的“设计研究者的核心价值在于判断与决策”高度一致。

研究三：剑桥大学工程设计中心（2023）《Design Intelligence: Augmenting Human Design Capability with AI》。该研究提出了“设计智能”的概念——人类设计师与AI系统之间形成一种分布式认知系统，人类的优势（情境理解、价值判断、创造性跳跃）与AI的优势（计算速度、模式识别、大规模数据处理）互补。研究发现，当前最有前景的应用方向不是全自动化设计，而是“交互式智能工具”——这正是本书“技能”概念的原型。

1.5.5 氛围编程的五项核心原则

基于以上讨论，我们总结出氛围编程的五项核心原则，这些原则将贯穿后续所有开发实践：

原则一：从最简单可用的版本开始。不要一开始就描述一个完美的、功能齐全的系统。先让AI生成一个能做一件事的脚本——能跑通、能看到输出，哪怕这件事很小。然后在此基础上逐步增加功能。这个“最小可行原型”（MVP）策略不仅降低了出错概率，也让每次迭代的反馈循环更短。

原则二：让AI解释它生成的代码。当你看到AI生成的代码中有一段你不理解，直接在对话中追问：“请解释一下这段代码在做什么，特别是第X行到第Y行。”AI会像一个耐心的导师一样逐行解释。这是氛围编程独有的学习优势——你一边做项目，一边通过对话建立对代码的理解。

原则三：善用AI的错误修复能力。在氛围编程中，报错不是停止的信号，而是继续对话的素材。把完整的错误信息（包括traceback——即报错信息中那堆文件路径和行号）原样粘贴给AI，它的修复建议通常非常精准。你不需要先理解错误信息在说什么——AI会理解。

原则四：保持人的判断力。AI生成的代码可能包含安全隐患（如硬编码API Key）、性能问题（如反复加载不必要的大模型）或逻辑错误（如否定词处理错误导致情感极性判断相反）。你需要保持审视——不是逐行审查语法，而是审查"这个输出是否合理"。当你发现输出不合理时，把观察到的现象描述给AI，让它定位和修复。

原则五：保护你的API Key。这是在所有开发实践中必须遵守的安全底线。永远不要把API Key直接写在代码里提交到公开仓库。使用环境变量或`.env`文件，并在`.gitignore`中将其排除。后续章节会详细演示如何做到这一点。

1.6 案例研究：用氛围编程开发访谈分析技能原型

本节通过一个完整的案例，将前面讨论的所有概念——技能、API、人机协同模式、氛围编程——串联起来。案例的主角是一位设计背景的研究生（我们称她为小林），她的编程基础几乎为零，但她有一个明确的需求：希望将自己近期完成的一组用户访谈稿件变得更加易于分析。

1.6.1 需求描述（氛围编程的第一步）

小林打开Cursor（一款AI编程编辑器），向其中的Claude模型发送了第一条消息：

> "我想做一个工具，帮助设计研究者分析用户访谈的文本。功能包括：
>
> 1. 读取访谈文字稿（txt文件）；
> 2. 自动做情感分析——标注每段话的情感是正面、负面还是中性；
> 3. 提取文本中的关键主题（3-5个）；
> 4. 生成一段200字以内的摘要。
>
> 请用Python实现，使用Gradio做界面，这样我可以用浏览器打开。"

注意这条提示词的结构：**目标陈述 → 功能列表（编号）→ 技术栈偏好。**这是一个典型的氛围编程提示词模板——它没有使用任何编程术语，只是清晰地列出了"做什么"（what），而不是"怎么做"（how）。AI的任务就是将这个what转化为how。

1.6.2 生成、运行与第一次迭代

AI（Claude）理解了需求，生成了约150行的Python代码。代码使用了`transformers`库的情感分析pipeline、`sklearn`的TF-IDF关键词抽取、以及Gradio的界面组件。

小林复制代码，保存为`interview_analyzer.py`，然后在终端运行。第一个错误出现了：

```
ModuleNotFoundError: No module named 'transformers'
```

小林将完整的错误信息复制粘贴给AI："运行后报这个错，怎么办？"AI回复："需要先安装依赖。请在终端运行以下命令：`pip install transformers gradio scikit-learn torch`。"小林照做，问题解决。

程序跑起来了，但情感分析的结果令人失望——几乎所有访谈段落都被判定为"中性"。小林再次与AI对话："情感分析的结果不够好，有没有更好的模型？"AI建议改用专门为社交媒体文本训练的`cardiffnlp/twitter-roberta-base-sentiment`模型，并提供了修改后的代码。替换后，结果显著改善。

这一轮交互揭示了氛围编程的一个核心模式：不是一次生成，而是通过持续对话不断逼近目标。小林扮演的是"产品经理"或"设计评审者"的角色——她提出需求，AI产出方案，她给出批判性反馈，AI修改方案。这与设计师向另一位设计师描述界面需求、对方给出初稿、然后反馈"这里信息层级不太对""那个按钮颜色可以更突出一点"的过程非常相似。

1.6.3 加入人机协同（增强模式）

小林的导师在审查这个原型后，提出了一个关键建议："主题提取的功能不要完全自动化。让AI给出建议主题和每个主题下的示例引语，然后让研究者可以修改主题名称、合并主题、移除不相关的主题。修改后再导出结果。"

小林将这个需求转化为自然语言指令：

> "请修改界面。在主题提取的部分：
>
> - AI先给出建议的主题（每个主题包含：主题名称、该主题下引用的3条示例引语）；
> - 在界面右侧加一个编辑区，让研究者可以：
>   - 修改主题名称；
>   - 拖拽合并两个主题；
>   - 删除不想要的主题；
>   - 添加新的主题。
> - 修改后的结果可以导出为JSON文件。"

这个改动从代码量上来说不小——涉及到界面布局重构、拖拽交互的实现、数据状态的维护。但从林的角度看，她只是用自然语言重新描述了一遍需求。AI生成了修改后的代码，虽然第一次运行时拖拽功能有一点bug，小林把bug现象描述给AI后，AI在第三次迭代时完全修复了。

这个改动正是1.4.2节所描述的"增强模式"的典型实现——AI提供建议（初始主题聚类），人类研究者进行审查和修改（编辑主题名称、合并、删除），最终产出融合了AI效率与人类判断的分析结果。

1.6.4 案例总结：氛围编程的成功要素

从小林的案例中，我们可以提炼出氛围编程的几个成功要素：

1. **精确描述**：使用"功能列表"式的结构，而不是模糊的"帮我做个分析工具"。
2. **渐进迭代**：先做最简单的版本（只有情感分析），跑通后再逐步添加主题提取、摘要生成、人机交互界面。
3. **善用报错**：把错误信息当成对话素材，而不是自己艰难地去搜索解决方案。
4. **理解边界**：AI擅长实现模式匹配类的功能（情感分析、文本聚类、界面生成），但不擅长需要深度领域知识判断的任务（如判定一个主题命名是否准确涵盖其下属引语的精髓）。正是后者，成为人类研究者在技能中发挥价值的地方。

1.7 实践工坊：第一个AI辅助编程作品

本章的最后一部分是一个动手工坊。读者将按照详细的步骤指南，完成第一个氛围编程项目：**用户反馈情感分析器。**

1.7.1 准备工作

在开始之前，请确认以下准备工作已完成：

- **硬件**：一台能够连接互联网的电脑（操作系统不限）。
- **AI编程工具**：从以下选项中选择一个：
  - **Cursor**（推荐）：cursor.com下载安装，内置多模型支持。
  - **Claude**：claude.ai网页使用，Claude 3.5 Sonnet及以上版本。
  - **Trae**：trae.ai，对中文友好。
  - **OpenCode**：开源工具，适合有一定基础的读者。
- **Python环境**：版本3.10或更高。打开终端/命令提示符，输入`python --version`或`python3 --version`验证。
- **API Key**：推荐DeepSeek API（platform.deepseek.com），注册后创建API Key。DeepSeek的定价极低，本节实操的API费用预计不超过人民币2元。

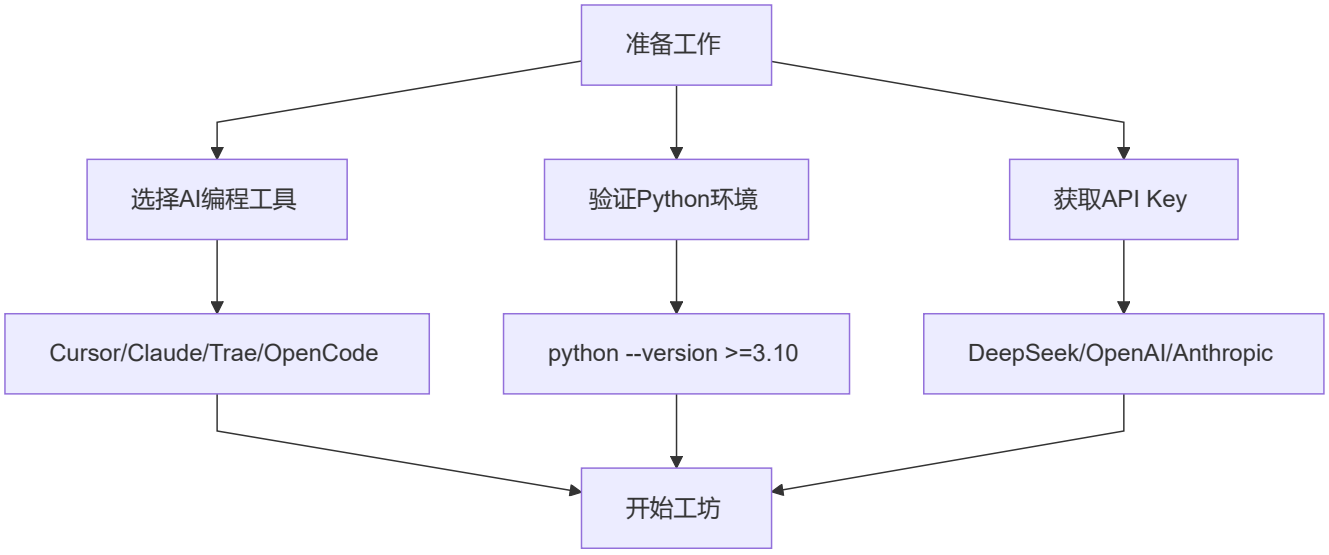


图1-5 实践工坊准备工作流程图

1.7.2 项目描述

项目名称：用户反馈情感分析器

功能：读取一段用户反馈文本，调用大语言模型API，输出以下结构化信息：

- 情感极性（正面/负面/中性）
- 情感强度（1-10分，1为最弱，10为最强）
- 关键句（1-3句）

为什么选择这个项目：它足够简单（30分钟内可完成），但同时包含了氛围编程的核心要素——调用API、处理文本、结构化输出、自测代码。这些要素在后续更复杂的技能开发中都会反复用到。

1.7.3 分步指南

第一步：编写提示词（5分钟）

打开Cursor，新建文件`sentiment_analyzer.py`，按`Ctrl/Cmd + K`打开AI对话窗口，输入以下提示词：

> 请帮我写一个Python脚本，功能如下：
>
> 1. 定义一个函数`analyze_feedback(text)`，接收一段用户反馈文本作为输入
> 2. 调用DeepSeek的API（使用OpenAI兼容接口，base_url是`https://api.deepseek.com`，模型名是`deepseek-chat`）
> 3. 让模型分析这段文本的情感极性（正面/负面/中性）和情感强度（1-10分），并提取1-3个关键句
> 4. 返回一个字典，包含：`sentiment`（情感极性）、`intensity`（强度）、`key_phrases`（关键句列表）
> 5. 在脚本的最后，用一段示例文本来测试这个函数，并将结果打印出来
>
> API Key使用环境变量`DEEPSEEK_API_KEY`来读取，不要硬编码在代码中。
>
> 需要安装的库请一并告诉我。

提示词结构拆解：这条提示词不是随意的描述，而是一份结构化的技术规格书：

- 第1条定义了函数签名（函数名、参数、入参类型）——告诉AI要生成一个可复用的函数，而不是一段散装的脚本。
- 第2条指明了基础设施（base_url、模型名）——如果不写，AI可能会编造一个不存在的端点。
- 第3条定义了语义输出规格（三分类+强度+关键句）——这本质上是一个编码框架。
- 第4条定义了数据契约（返回字典的结构）——保证输出可以被稳定的消费。
- 第5条要求自测代码——没有测试的交付是不可接受的。
- API Key的安全性要求是从一开始就要养成的习惯。

第二步：观察生成的代码（5分钟）

AI会生成一段代码。不急着运行，先阅读一遍基本结构。典型的代码结构包含四个部分：

```
# 第一部分：导入语句
import os
from openai import OpenAI

# 第二部分：初始化客户端
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

# 第三部分：核心函数
def analyze_feedback(text):
    response = client.chat.completions.create(
        model="deepseek-chat",
        messages=[
            {"role": "system", "content": "你是一个用户反馈分析助手，请严格按照JSON格式返回结果..."},
```

```
            {"role": "user", "content": text}
        ]
    )
    # 解析响应、构建字典、返回

# 第四部分：测试代码
if __name__ == "__main__":
    test_text = "这个产品的界面设计非常直观，我很喜欢，但是加载速度有点慢。"
    result = analyze_feedback(test_text)
    print(result)
```

作为设计背景的读者，你现在需要理解的是这个"骨架"——脚本导入了什么、初始化了什么、定义了哪个函数、最后怎么测试的。不必纠结于每行代码的细节，后续可以通过追问AI来逐步理解。

第三步：设置环境变量并运行（10分钟）

设置API Key（任选一种方式）：

- 临时设置（仅当前终端窗口有效）：
  - Windows命令提示符：`set DEEPSEEK_API_KEY=sk-你的key`
  - macOS/Linux终端：`export DEEPSEEK_API_KEY=sk-你的key`
- 永久设置（推荐学习使用`.env`文件，将在下一章介绍）

安装依赖：

```
pip install openai
```

运行脚本：

```
python sentiment_analyzer.py   # Windows/macOS均可
# 或
python3 sentiment_analyzer.py # macOS
```

预期输出（示例）：

```
{'sentiment': '正面', 'intensity': 7, 'key_phrases': ['界面设计非常直观', '我很喜欢']}
```

第四步：调试（当遇到错误时）

氛围编程的常态是第一次运行几乎必然遇到某种错误。以下是本节可能遇到的最常见错误及应对方式：

错误信息	可能原因	氛围编程解法
`ModuleNotFoundError: No module named 'openai'`	未安装openai库	将报错粘贴给AI，AI会建议运行`pip install openai`
`openai.AuthenticationError: Error code: 401`	API Key未设置或无效	检查环境变量是否正确设置；将报错粘贴给AI，AI会帮助排查

错误信息	可能原因	氛围编程解法
返回结果是文本而非字典	AI模型没有按JSON格式返回	向AI说明："请修改代码，让模型返回严格的JSON格式，并在Python中解析这个JSON"
`requests.exceptions.Timeout`	网络请求超时	向AI说明："请给API调用增加超时设置和重试逻辑，如果失败等待2秒再试，最多3次"

一个重要的心态提示：当你把报错信息粘贴给AI时，你不需要先理解报错在说什么。AI会帮你理解。你的任务是把问题精确地传达给AI，然后测试AI给出的解决方案。这就是氛围编程的"对话驱动调试"。

1.7.4 扩展练习

在成功运行基础版本后，尝试用氛围编程的方式增加一项功能：**批量分析**。

提示词示例：

> "请为我的`sentiment_analyzer.py`增加一个批量分析功能。新函数`analyze_batch(texts)`接收一个文本列表，返回一个字典列表，每个字典的结构与`analyze_feedback`相同。用3-5条示例文本测试这个批量功能。"

全程使用自然语言与AI对话完成，不要手写代码。完成后，记录下你使用的关键提示词和AI的每次回应。

1.8 研究方法池与团队协作

为帮助读者在后续章节中选择项目方向，本节简要介绍一个包含20种设计研究方法的"方法池"。每个方法都标注了4个评估维度：示例、可能用到的 AI 技术、API 候选、重构难度。这些维度的赋值将帮助判断该方法的AI重构难度与侧重点。

1.8.1 方法池概览

本课程的目标是把设计研究方法拆解成 AI 可理解、可执行、可组合的"Skill"，或进一步封装成可调用的 API，这本质上是在构建一个人机协同的"设计研究操作系统"。以下以设计流程的不同阶段为依据，为本课程的研究项目梳理了一份扩展后的可选重构清单，供各小组选题时参考。

1.8.1.1 用户洞察与研究类方法

这类方法最容易率先 AI 化，因为它们大量依赖文本、语音、行为数据的采集与解读。

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
用户深度访谈	**访谈智能分析 Skill**：上传访谈录音/文本→自动转录与说话人分离→NLP 情感与意图标记→关键主题聚类与语义网络图生成→输出摘要简报与代表性引语卡片。支持研究者点击主题节点深入查看原始片段。	语音转文字（Whisper）、说话人分离、文本情感分析、意图识别、主题建模（BERTopic/LDA）、LLM 链式提示摘要、文本嵌入与降维可视化	`/api/sentiment-analysis`（情感分析）、`/api/topic-extraction`（主题提取）、`/api/interview-summary`（访谈摘要生成）	■■
问卷调查与量表分析	**问卷洞察 Skill**：导入问卷数据（含封闭题与开放题）→自动完成描述统计与显著性检验→开放题文本聚类与观点聚合→抓取典型引用→生成图文并茂的洞察报告。支持交叉分析（如按人群维度下钻）。	自动统计检验（SciPy）、开放题文本嵌入（Sentence Transformers）、聚类（HDBSCAN）、LLM 开放题编码与引用抓取、自然语言生成报告	`/api/survey-stats`（问卷统计分析）、`/api/open-coding`（开放题编码）、`/api/cross-analysis`（交叉分析洞察）	■
用户观察/行为日志分析	**行为模式提取 Skill**：输入用户操作视频或行为日志→自动识别操作序列与关键动作→标记异常行为与痛点时刻（如反复点击、长时间停顿、任务放弃）→生成用户行为流程图与问题点分布图。	视频关键帧提取、目标检测/动作识别、序列模式挖掘、异常检测算法、LLM 行为解释、流程图自动生成	`/api/behavior-segmentation`（行为切分）、`/api/friction-detection`（摩擦点检测）、`/api/clickstream-analysis`（点击流分析）	■■■
人物角色（Persona）构建	**数据驱动 Persona 生成器 Skill**：输入用户调研数据集→自动无监督聚类形成用户类型→LLM 生成包含行为模式、目标、痛点、使用场景的角色卡片→支持新数据流入时动态更新角色属性。角色卡可导出为设计工具可读格式。	无监督聚类（HDBSCAN/K-Means）、维度降维与可视化、LLM 角色卡生成（模板化提示）、数据版本管理与增量更新	`/api/generate-persona`（角色生成）、`/api/update-persona`（角色更新）、`/api/user-segmentation`（用户分群）	■■

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
用户旅程地图（CJM）	**旅程地图自动生成 Skill**：输入多源数据（用户日志、访谈、反馈文本）→AI 识别用户行为阶段与触点→在每个触点上自动标注情绪峰值/低谷、痛点与机会点→生成结构化旅程 JSON，可导入 Miro/Figma 等工具进行可视化编辑。	序列模式识别、触点实体识别、情感弧线分析（基于文本/日志）、LLM 阶段命名与机会点推理、结构化 JSON 输出	`/api/touchpoint-emotion`（触点情绪分析）、`/api/journey-json`（旅程结构化数据生成）、`/api/opportunity-suggestion`（机会点建议）	▉▉▉
文化探针（Cultural Probes）	**探针解读 Skill**：批量导入用户回传的图文日记、照片、手绘等探针材料→多模态 AI 提取视觉主体、场景与情感基调→文本分析提取高频叙事主题→生成文化主题卡片与灵感图板，标记值得深入追问的异常信号。	多模态视觉理解（CLIP/GPT-4V）、图像标签与主体检测、图像情感/美学分析、文本主题聚类、异常信号检测	`/api/image-tagging`（图像标签提取）、`/api/visual-sentiment`（图像情绪分析）、`/api/theme-clustering`（主题聚类）	▉▉▉

1.8.1.2 问题定义与分析综合类方法

这些方法的核心在于从纷杂资料中形成"洞察模式"，AI 擅长从底层数据进行聚类、关联与溯因。

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
亲和图（Affinity Diagram）	**智能亲和图 Skill**：输入原始研究数据点（便利贴/笔记/引用）→AI 计算语义相似度并建议初始分组与集群标签→研究者通过拖拽界面调整分组→AI 实时反哺：重新命名未命名集群、检测离群点、建议合并相似集群。	文本嵌入（Sentence Transformers）、层次聚类/社区发现算法、LLM 集群自动命名、离群点检测、交互式界面（Gradio/Streamlit）	`/api/text-clustering`（文本聚类）、`/api/label-suggestion`（标签推荐）、`/api/outlier-detection`（离群点检测）	■■
主题分析（Thematic Analysis）	**协作式主题分析 Skill**：导入访谈/观察文本→AI 生成初始开放编码建议（可编辑/删除/合并）→研究者确认后，AI 将编码组织为主题网络→可视化呈现主题层级→研究者修改后，AI 自动搜索补充例证并检查主题内部一致性。	LLM 开放编码生成（少样本提示）、编码聚类与层级组织、知识图谱构建（主题网络）、语义搜索匹配引文、一致性检验（编码者信度模拟）	`/api/open-coding-assist`（开放编码辅助）、`/api/theme-consistency`（主题一致性检验）、`/api/quote-search`（引文搜索）	■■■
系统性文献/竞品分析	**竞品特征解构 Skill**：输入竞品素材（网页、文档、截图）→AI 自动提取功能列表、交互模式、视觉特征、技术架构与市场定位→生成结构化对比矩阵→支持按维度筛选与差异点高亮。可配置定时更新，监控竞品变化。	网页爬取与解析、多模态特征提取（文本+图像）、命名实体识别、LLM 结构化信息抽取、对比表生成、定时任务调度	`/api/competitor-extract`（竞品信息提取）、`/api/feature-matrix`（功能矩阵生成）、`/api/diff-highlight`（差异高亮）	■■

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
因果映射/系统图	**系统动力学建模 Skill**：输入问题描述与相关案例→AI 与研究者对话式地识别变量与因果关系→逐步构建因果环路图（Causal Loop Diagram）→标注正负反馈环→模拟假设情境下的系统行为变化。研究者可随时修正连接关系。	LLM 因果推理（链式提示）、图结构生成（NetworkX）、反馈环检测算法、对话式交互界面、简单的系统动力学模拟引擎	`/api/causal-graph`（因果图生成）、`/api/feedback-loop`（反馈环检测）、`/api/leverage-point`（杠杆点建议）	▉▉▉

1.8.1.3 创意生成与概念设计类方法

AI 在此扮演"灵感伙伴"和"组合加速器"的角色，尤其适合 Divergent Thinking 阶段。

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
头脑风暴/脑写	**AI 创意激发 Skill**：设定创意挑战后→基于 SCAMPER（替代/组合/适应/修改/他用/消除/重排）、随机词、类比、TRIZ 矛盾矩阵等多重规则→批量生成跨界概念火花→研究者标记/评分/扩展→AI 收敛为具有初步可行性的概念方向。	LLM 创意发散（温度调控+约束提示）、SCAMPER/TRIZ 规则引擎、类比生成（类比知识库）、语义新颖度评分、交互式收敛工作流	`/api/creative-stimulus`（创意刺激生成，可指定规则/领域）、`/api/analogy-match`（类比匹配）、`/api/novelty-score`（新颖度评分）	▉

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
形态学矩阵	**形态组合探索 Skill**：定义功能-载体矩阵（参数/子功能 × 可能的实现方式）→AI 自动生成组合空间→按合理性、新颖性、极端性进行三重过滤与排序→生成优先列表，附带冲突评估与可行性标注。支持研究者锁定某维度后探索其余维度组合。	组合优化与笛卡尔积生成、LLM 约束过滤与可行性/新颖性判断、冲突检测（基于 TRIZ 矛盾矩阵）、参数化矩阵输入界面	`/api/design-space`（设计空间生成）、`/api/combination-eval`（组合评估）、`/api/conflict-check`（冲突检测）	▉
概念草图/造型探索	**概念可视化 Skill**：输入设计语义关键词（如"有机、安全感、轻量化"）与可选风格参考图→生成式 AI 产出多种造型意象图板→按风格维度（流线型/几何/仿生等）自动分组→支持变体迭代（保留形态特征+改变材质/颜色/比例）→研究者挑选后生成图板报告。	Stable Diffusion/Midjourney/FLUX 文生图、ControlNet 可控生成、提示词工程（语义→视觉描述翻译）、图像相似度聚类、变体生成（img2img+提示词微调）	`/api/design-visualization`（设计语义出图）、`/api/style-exploration`（风格探索矩阵）、`/api/variant-generation`（变体生成）	▉▉

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
设计虚构/未来情境	**未来世界构建 Skill**：输入趋势信号（如人口老龄化+AI 陪伴普及）→AI 生成连贯的虚构未来元素：新闻头条、产品开箱评测、社交媒体抱怨、监管文件草案→研究者沉浸其中讨论→AI 根据讨论产出设计机会简报与新兴用户需求假设。	LLM 叙事生成（世界观一致性约束）、时序推理（趋势到情境的推演）、角色化文本生成（不同文体）、情感一致性控制	`/api/scenario-generator`（情境生成）、`/api/future-news`（虚构新闻生成）、`/api/design-brief`（机会简报生成）	▉▉

1.8.1.4 原型与评估类方法

转向"测试-学习"循环，AI 可自动化数据采集、行为分析和异常检测。

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
可用性测试（轻声思维法）	**可用性测试分析 Skill**：上传测试录像→自动识别任务边界并切段→检测错误操作与困惑行为→同时分析面部表情与语音情绪→生成高亮问题时刻合集（视频片段+问题标注）→输出问题分布热力图与严重度排序。研究者可点击高亮片段快速回溯原始录像。	视频帧分析+UI 状态识别、面部表情识别（DeepFace/OpenFace）、语音情绪分析（OpenSmile）、任务步骤识别（基于操作序列）、LLM 问题分类与严重度判断	`/api/usability-highlights`（问题时刻提取）、`/api/emotion-timeline`（情绪时间线）、`/api/task-segmentation`（任务自动切段）	▉▉▉

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
A/B 设计与多变量测试	**实验生成与解读 Skill**：输入待测变量与假设→AI 建议 A/B/n 实验设计方案→生成测试刺激材料草稿→实验结束后自动进行统计检验→生成包含显著性结论与质性行为解释的综合报告，标注统计与实务显著性的差异。	实验设计逻辑引擎（因子/水平自动配置）、LLM 测试材料生成、自动统计检验与效应量计算、自然语言实验报告生成	`/api/experiment-design`（实验方案生成）、`/api/ab-test-report`（A/B 报告生成）、`/api/test-material`（测试材料生成）	▉▉
设计评论/启发式评估	**启发式评审助手 Skill**：上传界面截图或设计稿→AI 逐条对照尼尔森十大启发性原则（或自定义原则集）→自动检测潜在违规点（如一致性、错误预防、帮助文档等维度）→为每个问题标注违反的原则、位置、严重度和改进建议→生成优先级排序的评审报告。支持批量审查与版本对比。	多模态视觉理解（GPT-4V/Claude Vision）、UI 元素检测与识别、启发式原则知识库与审查链、LLM 评审意见生成（原则→检测→描述→建议）、版本差异对比	`/api/heuristic-review`（启发式评审）、`/api/ui-audit`（界面扫描审核）、`/api/consistency-check`（一致性检查）	▉▉

1.8.1.5 战略与前瞻类方法

这类方法需要处理大量弱信号，AI 的 pattern recognition 能辅助捕捉趋势。

方法	可产出的 Skill 示例	可能用到的 AI 技术	API 候选	重构难度
趋势扫描与信号侦测	**趋势雷达 Skill**：配置监控领域与技术关键词→自动采集全网信息（新闻/论文/专利/社交媒体）→AI 聚类新兴主题→识别变化加速的信号（如讨论频率突增）→标记信号强弱与成熟度→周期性生成趋势简报与早期信号预警。研究者可钻取到信号源头。	网络爬虫/RSS/API 数据采集、文本嵌入与增量聚类（River/BERTopic）、时间序列异常检测（信号加速识别）、LLM 趋势主题命名与简报生成	`/api/trend-monitor`（趋势监控）、`/api/signal-detect`（弱信号检测）、`/api/trend-briefing`（趋势简报生成）	▉▉▉
SWOT/PEST 分析	**环境分析报告生成器 Skill**：输入产业/产品名称与分析范围→AI 从公开报告、新闻、行业数据中多源检索相关证据→按 S/W/O/T 与 P/E/S/T 象限自动归类与填充→为每个条目标注信息来源与可信度权重→生成结构化 SWOT/PEST 草案，供专家审核与修正。支持不同时间版本的趋势对比。	多源信息检索（联网搜索/知识库）、LLM 证据提取与归类、来源可信度评估、象限结构化填充（含引用标注）、版本差异对比	`/api/swot-generator`（SWOT 生成）、`/api/pest-analysis`（PEST 分析）、`/api/evidence-extract`（证据提取与溯源）	▉▉
价值主张画布	**价值主张对齐 Skill**：输入前期用户研究数据（痛点/收益/任务）与产品特性列表→AI 自动建立痛点-解决方案、收益-创造收益的配对映射→检测未覆盖的用户需求与无对应需求的产品特性→生成价值主张一致性评分与改进建议→输出可视化对齐矩阵。	LLM 语义匹配（用户需求↔产品特性）、覆盖度计算、差距分析、自然语言改进建议生成、对齐矩阵可视化	`/api/value-alignment`（价值主张对齐度分析）、`/api/gap-analysis`（需求-功能差距分析）、`/api/pain-point-match`（痛点-方案匹配）	▉

表1-4 设计研究方法池及评估维度

说明

1. **重构难度标注**：▉ 为相对容易上手（主要依赖 LLM 文本处理，技术栈简单），▉▉ 为中等难度（涉及多步流水线或特定模型），▉▉▉为有挑战性（需要多模态处理、实时系统或复杂算法）。建议各小组根据自身兴趣和技术基础选择。
2. **API 候选关系**：每个方法下的多个 API 候选代表了 Skill 中可被拆分为独立服务的子功能模块。并非所有 API 候选都必须实现——小组可根据 Skill 的设计，选择其中 1-2 个最核心的进行封装。
3. **人机协同节点**：所有 Skill 示例均预设了研究者介入的关键环节（如确认聚类方案、修正评审结果、评分创意方向），这是设计研究 AI 工具区别于全自动工具的核心特征，也是本课程需要学生重点设计的内容。
4. **项目管理理念**：每位学生只需深入掌握一个方法的重构过程，而前 3 周的理论与 Vibe Coding 训练确保学生能够将自然语言思维转化为可工作的 AI 工具。整个过程强调"Design Prompts, not just Code"——通过描述需求、分析、修正来驱动开发，最大化降低编程门槛，聚焦设计研究方法本身的转化逻辑。

1.8.2 选题建议

基于本章的内容，读者在选择自己的项目方法时，可以考虑以下几点：

1. **选择你真正感兴趣的方法。**一个8周的项目需要你反复与这个方法打交道——如果你对它本身没有热情，很难保持高质量的迭代。
2. **评估你的数据获取能力。**访谈、问卷、可用性测试等方法需要真实的原始数据。如果你手头有之前项目积累的数据，这是巨大优势。如果没有，可以使用模拟数据集或AI生成的合成数据（这在氛围编程流程中完全可以接受）。
3. **考虑你的技术信心。**如果对编程完全零基础，可以从难度较低的方法开始（如启发式评估、形态学矩阵）。但请记住，无论选择哪种方法，开发工具都是氛围编程——技术门槛已经被大幅降低了。

1.9 本章小结

本章作为《智能设计系统》教材的开篇，完成了以下工作：

第一，回顾了设计研究的五个经典阶段——用户洞察、问题定义、创意生成、原型评估、战略前瞻，并系统分析了AI技术在每个阶段带来的重构机遇。我们的核心命题是：设计研究方法重构的本质不是AI替代人，而是从"工具辅助人"转向"人机协同"。

第二，引入了"技能"（Skill）这一核心概念，明确了方法与技能的根本区别，讨论了默会知识的边界问题，建立了技能的三个基本属性（明确输入输出、可复用逻辑、人机协同设计），并将技能划分为原子、流程、复合三个层级，给出了层级选择的决策逻辑。

第三，阐释了API的概念及其与技能的关系，说明了将技能封装为API在可嵌入性、可组合性、可规模化三方面的价值。

第四，识别并详细阐述了人机协同的四种模式——自动化模式、增强模式、激发模式、代理模式，并给出了综合应用的决策指南。

第五，介绍了氛围编程这一开发范式，从定义、适用性、核心原则到实证研究支撑，构建了理论框架；并通过访谈分析技能原型的完整案例，演示了氛围编程的全过程。

第六，提供了第一个动手工坊的详细步骤指南，帮助读者完成一个可工作的情感分析器。

最后，给出了方法池概览和选题建议，为后续章节的项目实践做好准备。

通过本章的学习，读者应该已经具备了理解后续章节所需的概念工具和实践信心。从“方法使用者”到“技能设计者”的转变，始于本章所建立的语言与认知框架。

思考与练习

一、概念理解题

1. 请用自己的语言解释“方法”与“技能”的根本区别，并给出一个设计研究之外的类比。
2. 什么是波兰尼所说的“默会知识”？请结合你自己的一个设计实践经历，举例说明该实践中哪些部分属于显性知识，哪些属于默会知识。
3. 人机协同的四种模式（自动化、增强、激发、代理）分别适合什么场景？请各举一个设计研究中的例子。
4. “氛围编程”的五项核心原则是什么？你认为哪一项对设计背景的学习者最重要？为什么？

二、分析应用题

5. 阅读以下用户访谈片段（虚构），尝试人工标注情感极性和关键句。然后，运行你在1.7节中完成的情感分析器，对比结果。如果结果不一致，分析可能的原因。

> “我用了这个APP大概两周吧。说实话，界面确实挺漂亮的，这个得承认。但是功能太少了，我想要的导出功能根本没有。而且经常闪退，尤其是晚上用的时候。客服回复倒是挺快的，但每次都说‘感谢反馈，我们会在后续版本优化’，也没见真的优化过。”

6. 假设你要将“亲和图”方法重构为一个AI技能。请为以下环节选择人机协同模式，并说明理由：a) 便利贴文字识别与数字化；b) 初始聚类分组；c) 聚类标签命名；d) 最终分组方案的确认与导出。

三、实践与反思题

7. 完成1.7节中的扩展练习（批量分析功能）。记录你与AI的完整对话历史（提示词 + AI回应 + 你对代码的修改）。在反思中回答：AI在哪些环节的表现超出预期？哪些环节需要你反复引导？你认为这种“对话式开发”与传统的“学习语法→编写代码”模式相比，优势和局限性分别是什么？
8. 回想你过去完成的一个设计研究项目（可以是课程作业、实习项目或毕业设计）。基于本章的方法池评估维度，为该项目中使用的主要方法填写下表（复制到本地填写）：方法名称、数据量（低/中/高/很高）、结构化潜力（低/中/高/很高）、交互复杂度（低/中/高）。然后写出300-500字的反思：如果让你用本章介绍的AI技能方法重新做这个项目，你会改变哪些环节？你会设计什么样的技能？你希望保留哪些环节的自己判断？

参考文献

[1] Polanyi, M. (1966). *The Tacit Dimension*. University of Chicago Press.

[2] Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*.

[3] Dell’Acqua, F., McFowland, E., Mollick, E. R., Lifshitz-Assaf, H., Kellogg, K., Rajendran, S., … & Lakhani, K. R. (2023). Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of AI on Knowledge Worker Productivity and Quality. *Harvard Business School Working Paper*, No. 24-013.

[4] Girotra, K., Meincke, L., Terwiesch, C., & Ulrich, K. T. (2023). Ideas are Dime a Dozen: Large Language Models for Idea Generation in Innovation. *arXiv preprint arXiv:2307.02008*.

[5] Karpathy, A. (2024). Vibe Coding: A New Way to Program. *Personal Blog / X Post*.

[6] Maiden, N., & Jones, S. (2023). Design Intelligence: Augmenting Human Design Capability with AI. *Cambridge Engineering Design Centre Technical Report*.

[7] Beyer, H., & Holtzblatt, K. (1998). *Contextual Design: Defining Customer-Centered Systems*. Morgan Kaufmann.

[8] Braun, V., & Clarke, V. (2006). Using Thematic Analysis in Psychology. *Qualitative Research in Psychology*, 3(2), 77-101.

[9] Nielsen, J. (1994). Heuristic Evaluation. In *Usability Inspection Methods* (pp. 25-62). John Wiley & Sons.